Engaging Howto Recipes MPI

Message Passing Interface (MPI)

Message Passing Interface (MPI) is a standard designed for data communication in parallel computing. The MPI standard defines useful library functions/routines in C, C++, and Fortran. Python interface is available for MPI.

There are several MPI implementations, such as `OpenMPI`, `MPICH`, `MVAPICH`, and `Intel MPI`, which work with Infiniband network for high-bandwidth data communication.

MPI modules

There are OpenMPI modules available on the cluster. Before building or runrning your MPI programs, load the modules of a `gcc` compiler and an `openmpi` libraries to set up environment varialbes.

There are two different operations systems (OS) on the cluster: CentOS 7 and Rocky 8. For CentOS 7 nodes, load these modules,

```
module load gcc/6.2.0 openmpi/3.0.4
```

or

```
module load gcc/9.3.0 openmpi/4.0.5
```

For Rocky 8 nodes, load these modules,

```
module use /orcd/software/community/001/modulefiles
module load gcc/12.2.0 openmpi/4.1.4-pmi-ucx-x86_64
```

If you need to run MPI programs with GPUs, load these modules instead,

```
module load gcc/12.2.0 openmpi/4.1.4-pmi-cuda-ucx-x86_64
```

Note

Load a `gcc` module first, then the openmpi modules built with this `gcc` will be shown in the output of `module avail` and can be loaded.

Build MPI programs

This section will be focused on building MPI programs in C or Fortran. Python users can refer to this page for using the `mpi4py` package.

Most MPI software should be built from source codes. First, download the package from the internet. Load one of the OpenMPI modules mentioned above. A typical building process is like this,

```
./configure CC=mpicc CXX=mpicxx --prefix=</path/to/your/installation>
make
make install
```

Create an install directory and assign its full path to the flag `--prefix`. This is where the binaries will be saved.

Widely-used MPI software includes `Gromacs`, `Lammps`, `NWchem`, `OpenFOAM` and many others. The building process of every software is different. Refer to its official installation guide for details.

Side note: MPI binaries

Some MPI software are provided with prebuilt binaries only. In this case, download the binaries compatible with the `linux` OS and the `x86_64` CPU architecture. If possible, try to choose an OpenMPI version, that the binary was built with, as close as possible to that of a module on the cluster. This type of MPI software includes `ORCA`.

Spack is a popular tool to build many software packages systematically on clusters. It makes building processes convenient in many cases. If you want to use Spack to build your software package on the cluster, refer to the recipe page for Spack.

If you develop your MPI codes, the codes can be compiled and linked like this

```
mpicc -O3 name.c -o my_program
```

or

```
mpif90 -O3 name.f90 -o my_program
```

This will create an executable file named `my_program`. Prepare a GNU Makefile to build programs with multiple source files.

MPI jobs

MPI programs are suitable to run on multiple CPU cores of a single node or on multiple nodes.

Here is an example script (e.g. named `job.sh`) to run an MPI job using multiple cores on a single node.

```
#!/bin/bash
#SBATCH -p sched_mit_hill
#SBATCH -t 30
#SBATCH -N 1
#SBATCH -n 8
#SBATCH --mem=10GB

srun hostname
module load gcc/6.2.0 openmpi/3.0.4
mpirun -n $SLURM_NTASKS my_program
```

This job requests 8 cores (with `-n`) and 10 GB of memory (with `--mem`) on 1 node (with `-N`) for 30 minutes (with `-t 30`). The `-n` flag is the same as `--ntasks`. The specified value is saved to the variable `SLURM_NTASKS`. In this case, the requested number of cores is equal to `SLURM_NTASKS`. The command `mpirun -n $SLURM_NTASKS` ensures that each MPI task runs on one core.

The command `srun hostname` is to check if the correct number of cores and nodes are assigned to the job. It is not needed in production runs.

Side note: partitions and modules

The modules used in the above example is for the CentOS 7 OS, which works for these partitions: `sched_mit_hill`, `newnodes`, and `sched_any`. If using a partition with the Rocky 8 OS, such as `sched_mit_orcd`, change the modules accrodingly (see the first session). These are public partitions that are avaiable to most users. Many labs have partitions for their purchased nodes.

Submit the job with the `sbatch` command,

```
sbatch job.sh
```

To run an MPI job on multiple nodes, refer to the following exmaple script.

```
#!/bin/bash
#SBATCH -p sched_mit_hill
#SBATCH -t 30
#SBATCH -N 2
#SBATCH --ntasks-per-node=8
#SBATCH --mem=10GB

srun hostname
module load gcc/6.2.0 openmpi/3.0.4
mpirun -n $SLURM_NTASKS my_program
```

This job requests 2 nodes with 8 cores and 10 GB of memory per node. In this case, the total number of cores (saved to `SLURM_NTASKS`) is equal to the number of nodes (saved to `SLURM_NNODES`) times the number of cores per node (saved to `SLURM_NTASKS_PER_NODE`). The command `mpirun -n $SLURM_NTASKS` ensures that MPI tasks are distributed to both nodes and each MPI task runs on one core.

Alternatively, users can specify the number of cores per node using an OpenMPI option like this `mpirun -npernode $SLURM_NTASKS_PER_NODE my_program`.

If replacing `--ntasks-per-node=8` with `-n 16` in the above script, the job will request 16 cores on 2 nodes, but it is not always the case that there are 8 cores per node. For example, there may be 7 cores on one node and 9 cores on another, or 1 core on one node and 15 cores on another, etc, depending on the current available resources on the cluster.

Computing resources for MPI jobs

To get a better idea on how many nodes, cores and memory should be requested, users need to consider the following two questions.

First, what resources are available on the cluster? Use this command to check node and job info on the cluster, including the constraint associated with OS (`%f`), the nubmer of CPU cores (`%c`), the memory size (`%m`), the wall time limit (`%l`), and the current usage status (`%t`).

```
 sinfo -N -p sched_mit_hill,newnodes,sched_any,sched_mit_orcd -o %f,%c,%m,%l,%t
|grep -v drain
```

On typical nodes of the cluster, the number of cores per node varies from 16 to 128, and the memory per node varies from 63 GB to 515 GB.

To obtain a good performance of MPI programs, it is recommended to request all physical CPU cores and memory on each node. For example, request two nodes with 16 physical cores per node and all of the memory like this,

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=16
#SBATCH --mem=0
```

As MPI is a distributed-memory parallelism, sometimes it is good to use the `--mem-per-core` flag assigning a certain amount of memory to each core. The total memory is increased with the number of cores in this case. Double check that the total memory fits in the maximum memory of a node to avoid failed jobs.

Second, what is the speedup of your MPI program? According to Amdahl's law, well-performing MPI programs are usually speeded up almost linearly as the number of cores is increased until it is saturated at some point. In practice, try to run testing cases investigating the speedup of your program, and then decide how many cores are needed to speed it up efficiently. ***Do not increase the number of cores when speedup is poor.***

Note

After a job starts to run, execute the command `squeue -u $USER` to check which node the job is running on, and then log in to the node with `ssh <hostname>` and execute the `top` command to check how many CPU cores are being used by the program and what the CPU efficiency is. The efficiency may vary with the number of CPU cores. Try to keep your jobs at a high efficiency.

Hybrid MPI and multithreading jobs

MPI programs are based on distributed-memory parallelism, that says, each MPI task owns a faction of data, such as arrays, matrices, or tensors. In contrast to MPI, the multithreading technique is based on a shared-memory parallelism, in which data is shared by multiple threads. A common implementation of the multithreading technique is OpenMP. For Python users, the `numpy` package is based on C libraries, such as Openblas, which are usually built with OpenMP.

Side note: OpenMP

OpenMP is an abbreviation of Open Multi-Processing. It is not related to OpenMPI.

Some programs are designed in a hybrid scheme such that MPI and OpenMP are combined to enable two-level parallelization. Set MPI tasks and OpenMP threads in hybrid programs based on the following equation,

```
(Number of MPI Tasks) * (Number of Threads) = Total Number of Cores          (1)
```

Side note: hyperthreads

There are two or multiple hyperthreads on each CPU core in modern CPU architecture. The hyperthread technique is turned off for most nodes of this cluster, but it may be turned on for some nodes as requested by the owner labs. In the case that there are two hyerthreads per physical core, the right side of the equation should be `2 * (Total Number of Cores)` instead.

One way to run hybrid progmrams in Slurm jobs is to use the `-n` flag for the number of MPI tasks and the `-c` flag for the number of threads. The follwing is a job script that runs a program with 2 MPI tasks and 8 threads per task on a node with 16 cores.

```
#!/bin/bash
#SBATCH -p sched_mit_hill
#SBATCH -t 30
#SBATCH -N 1
#SBATCH -n 2
#SBATCH -c 8
#SBATCH --mem=10GB

module load gcc/6.2.0 openmpi/3.0.4
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
mpirun -n $SLURM_NTASKS my_program
```

The `-c` flag is the same as `--cpus-per-task`. The specified value is saved in the variable `SLURM_CPUS_PER_TASK`. In this case, the total number of cores equals `SLURM_NTASKS * SLURM_CPUS_PER_TASK`, that is 16.

The built-in environment variable `OMP_NUM_THREADS` is used to set the number of threads for an OpenMP program. Here it is equal to `SLURM_CPUS_PER_TASK`. The number of MPI tasks is set to be `SLURM_NTASK` in the `mpirun` command line, therefore, the nubmer of MPI tasks times the number of threads equals the total number of CPU cores.

Users only need to specify the numbers following Slurm flags `-n` and `-c`, for example, `-n 4 -c 4` or `-n 8 -c 2`, keeping the product unchanged, then the MPI tasks and threads are all set automatically.

Similarly, here is an exmple script to request two nodes,

```
#!/bin/bash
#SBATCH -p sched_mit_hill
#SBATCH -t 30
#SBATCH -N 2
#SBATCH --ntasks-per-node=2
#SBATCH -c 8
#SBATCH --mem=0

module load gcc/6.2.0 openmpi/3.0.4
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
mpirun -n $SLURM_NTASKS my_program
```

In this case, the total number of cores is equal to `SLURM_NNODES * SLURM_NTASKS_PER_NODE * SLURM_CPUS_PER_TASK`, that is `2 * 2 * 8 = 32`. The job will run 4 MPI tasks (i.e. 2 tasks per node) and 8 threads per task. Equation (1) is satisfied as `4 * 8 = 32`.

As mentioned in the previous section, it is recommended to run testing cases to determine the values for the flags `-N`, `-n` and `-c` to obtain a better performance.

There is another way to submit jobs for hybrid programs, in which the `-c` flag is not used at all. For example, it also works like this,

```
#!/bin/bash
#SBATCH -p sched_mit_hill
#SBATCH -t 30
#SBATCH -N 1
#SBATCH -n 16
#SBATCH --mem=10GB

module load gcc/6.2.0 openmpi/3.0.4
export OMP_NUM_THREADS=8
MPI_NTASKS=$((SLURM_NTASK / $OMP_NUM_THREADS))
mpirun -n $MPI_NTASKS my_program
```

This job requests 16 CPU cores on 1 node and runs 2 MPI tasks with 8 threads per task, so equation (1) is satisfied as `2 * 8 = 16`. In this case, users need to set the values for the Slurm flag `-n` and the variable `OMP_NUM_THREADS`.

February 11, 2026